ملخص لأهم المفاهيم والخصائص في كود مساعد Gemini AI

يقدم هذا الملخص أبرز المفاهيم والتقنيات الأساسية التي يجب استخلاصها من الكود البرمجي، والتي تشكل جوهر بناء المساعد الذكي التفاعلي.

1. معالجة اللغات الطبيعية (Natural Language Processing - NLP)

العمود الفقري لفهم الكود هو استخدام مكتبة **NLTK** (Natural Language Toolkit) [1] لمعالجة النصوص. تتضمن المفاهيم الرئيسية هنا:

- **تقسيم الكلمات (Tokenization):** تحويل الجمل إلى وحدات أصغر (كلمات أو رموز) باستخدام `nltk.word_tokenize()` . هذه هي الخطوة الأولى في تحليل أي نص.
- **تصغير الكلمات (Stemming):** تقليل الكلمات إلى جذرها أو شكلها الأساسي (مثل تحويل "running" إلى "run") باستخدام `LancasterStemmer` . يساعد هذا في توحيد الكلمات وتقليل حجم المفردات، مما يحسن أداء النموذج.
- **كيس الكلمات (Bag of Words - BoW):** تمثيل نصي بسيط حيث يتم تحويل الجملة إلى متجه رقمي يمثل تكرار الكلمات (أو وجودها) في مفردات محددة. هذا التمثيل هو المدخل الرئيسي لنموذج التعلم الآلي.

2. التعلم الآلي والشبكات العصبية (Machine Learning & Neural Networks)

يستخدم الكود مكتبة **TensorFlow** [2] ، وبالتحديد واجهة Keras البرمجية، لبناء وتدريب نموذج تعلم آلي قادر على تصنيف نوايا المستخدم:

- **الشبكات العصبية الاصطناعية (Artificial Neural Networks - ANN):** يتم بناء نموذج شبكة عصبية تسلسلية (Sequential model) تتكون من طبقات `Dense` (متصلة بالكامل).
- **طبقات الإدخال والإخراج (Input & Output Layers):** طبقة الإدخال تستقبل متجه "كيس الكلمات"، بينما طبقة الإخراج تستخدم دالة التنشيط `softmax` لتصنيف النية إلى واحدة من الفئات المحددة (مثل "تحية"، "وداع"، "سؤال").
- **التدريب (Training):** يتم تدريب النموذج باستخدام بيانات الأنماط والنوايا (`training` و `output`) لتعلم العلاقة بين مدخلات المستخدم والنية الصحيحة. يتم حفظ النموذج المدرب (`chatbot_model.h5`) لإعادة الاستخدام.
- **التنبؤ (Prediction):** بعد التدريب، يمكن للنموذج التنبؤ بنية أي جملة جديدة يدخلها المستخدم بناءً على ما تعلمه.

3. استمرارية البيانات (Data Persistence)

لتحسين الكفاءة وتجنب إعادة المعالجة والتدريب في كل مرة، يستخدم الكود تقنيات استمرارية البيانات:

- **حفظ وتحميل البيانات المعالجة:** يتم استخدام مكتبة `pickle` لحفظ البيانات المعالجة مسبقًا (مثل الكلمات الفريدة، التصنيفات، بيانات التدريب) في ملف `data.pickle` . هذا يسمح بتحميلها بسرعة عند تشغيل التطبيق بدلاً من إعادة معالجتها من البداية.
- **حفظ وتحميل النموذج المدرب:** يتم حفظ نموذج TensorFlow المدرب في ملف `chatbot_model.h5` . هذا يعني أن النموذج لا يحتاج إلى إعادة التدريب في كل مرة يتم فيها تشغيل التطبيق، مما يوفر الوقت والموارد.

4. تطوير تطبيقات الويب التفاعلية (Interactive Web Application (Development

تُستخدم مكتبة **Streamlit** `3` لإنشاء واجهة المستخدم الرسومية (GUI) للمساعد الذكي، مما يجعله تطبيق ويب تفاعليًا:

- **بناء الواجهة بسرعة:** Streamlit يسمح بإنشاء واجهات ويب تفاعلية باستخدام كود بايثون بسيط، مما يسهل عرض المساعد الذكي.
- **إدارة حالة الجلسة (Session State):** تُستخدم `st.session_state` لتخزين الرسائل السابقة في المحادثة، مما يسمح للمساعد الذكي بتذكر سياق الحوار.
- **تخصيص المظهر (Custom Styling):** يمكن حقن كود CSS مخصص باستخدام `st.markdown` لتغيير مظهر التطبيق، مثل ألوان الخلفية والنصوص، وتطبيق تأثيرات بصرية مثل تدرج الألوان على العنوان.
- **عناصر واجهة المستخدم التفاعلية:** مثل أزرار الاختيار (`st.radio`) وحقول إدخال الدردشة (`st.chat_input`)، التي تمكن المستخدم من التفاعل مع التطبيق.

5. منطق الروبوت المحادثة (Chatbot Logic)

يجمع الكود بين المفاهيم المذكورة أعلاه لتشغيل منطق الروبوت المحادثة:

- **تصنيف النوايا (Intent Classification):** بناءً على إدخال المستخدم، يقوم النموذج بتحديد النية المحتملة (مثل "تحية"، "وداع"، "سؤال عن الطقس").
- **توليد الردود (Response Generation):** بمجرد تحديد النية بثقة كافية، يتم اختيار رد مناسب وعشوائي من قائمة الردود المحددة مسبقًا لتلك النية.
- **التفاعل مع المستخدم:** يتم عرض رسائل المستخدم وردود المساعد الذكي بشكل ديناميكي في واجهة الدردشة، مع إضافة لمسات مثل مؤشر التحميل والتأخير الزمني لجعل التجربة أكثر طبيعية.

المراجع

[1] NLTK (Natural Language Toolkit). (n.d.). Retrieved from

[2] TensorFlow. (n.d.). Retrieved from
[3] Streamlit. (n.d.). Retrieved from